Design and Implementation of a Compiler with Machine-Learning-Guided Optimization Selection

Kedar Sedai

***Abstract*—This paper documents the complete design, implementation, and evaluation of MiniLang, a research compiler for a minimal statically typed imperative language. MiniLang was build from scratch in C++17 to support end-to-end study of compiler construction and machine-learning-guided optimization. The system implements a classical multi-phase pipeline:hand-written lexical analysis, recursive-descent parsing, two-pass semantic analysis, lowering to a flat High-level Intermediate Representation(HIR), configurable rule-based optimizations, textual LLVM IR emission, and an ML-ready optimization advisor with evaluation harness.**

We describe every compilation phase in implementation-level detail, including token design, abstract syntax tree (AST) structure, semantic rules, HIR instruction set, three optimization passes, LLVM code generation strategy, and a feature-driven advisor that selects optimization pipeline using heuristics, JSON plans or external Python interface. Experimental results in benchmark programs show that optimization reduces the HIR instruction count by up to 63% ($22 \rightarrow 8$ instruction) and LLVM IR size by 41% ($34 \rightarrow 20$ lines) on constant-heavy workloads, while preserving correct runtime behavior(verified by process exit codes.) Minilang is intended as a reproducible research and teaching traditional compiler engineering and modern ML-for-compilers research

***Index Terms*—Compilers, intermediate representation, LLVM, static analysis, code optimization, machine learning, pass ordering**

I. Introduction

A. Motivation

Optimizing compilers transform source programs through dozens of analysis and transformation passes. Deciding which passes to run, and in what order has been tuned manually for decades in systems such as GCC and LLVM[1],[2]. Recently, machine learning(ML) has been applied to predict profitable compiler decisions-including pass ordering, heuristics flags, and inlining thresholds-from program features[3]-[6].However, experimenting with ML-guided optimization typically requires navigating industrial-strength compiler infrastructures whose complexity obscures the relationship between front-end structure, IR design, and optimization outcomes.

Minilang addresses this by providing a complete but minimal compiler where every phase is inspectable, independently executable, and instrumented for measurement. This project follows a staged research methodology:

- Language design - define a small typed language sufficient for control flow, functions, and arithmetic.
- Front-end construction- implement lexer, parser, and semantic analyzer with explicit error reporting.
- IR design - lower AST to flat HIR with explicit temporaries and structured control flow.
- Rule-based optimization - implement classic local optimizations at HIR level.
- Code generation - emit human-readable LLVM IR for execution via Clang.
- ML advisor integration - extract static features, select pass pipelines, and evaluate against baselines.

B. Research Questions

This works investigates:

- RQ1: Can a minimal imperative language support a full, pedagogically transparent compilation pipeline comparable to textbook architecture [1],[7]?
- RQ2: Is a flat HIR with explicit temporaries a suitable layer for both rule-based optimization and feature extraction?
- RQ3: Can static HIR metrics drive selective application of optimization passes with measurable IR reduction?

C. Contributions

- Formal specification of the MiniLang language (lexical, syntactic, and semantic rules).
- Complete architectural documentation of a C++17 compiler with six standalone driver tools.
- HIR specification with lowering algorithms for expressions, conditionals, and loops.
- Three HIR optimizations with fixed-point iteration and statistics.
- ML-ready advisor framework:14-dimensional feature vector, pass, registery, heuristic/JSON/Python modes, and minilang_eval harness.
- Empirical evaluation demonstrating IR reduction and correctness on benchmark programs.

D. Paper Organization

Section II reviews related work. Section III defines MiniLang. Section IV presents system architecture. Sections V-X detail each compilation phase. Section XI covers the ML advisor. Section XII presents evaluation. Section XIII-XIV discuss limitations and conclude.

II. Related Work

A. Compiler Construction Foundations

The classic compiler pipeline- lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization and code generation- is established in Aho et al.[1] and Cooper and Torczon [2]. MiniLang adopts this structure verbatim, using a hand-written scanner and recursive-descent parser rather than generator tools(Flex/Bison) to maximize transparency for readers and students.
Appel [8] and Muchnick [9] emphasize the role of intermediate representation in decoupling front-end language features from back-end code generation. Our HIR follows the principle: it is lower than AST(flat instruction list) but higher than LLVM (no SSA no target- specific details).

B. LLVM as a Back-end

Lattner and Adve introduced LLVM as a modular compilation framework with SSA-based intermediate representation and aggressive optimization infrastructure [2]. Rather than linking against the LLVM C++ API initially, MiniLang emits textual LLVM IR (.ll files), which Clang consumes for native code generation, This approach mirrors the LLVM Kaleisoscoe tutorial [10] and keeps the research artifact readable in published papers.

C. Machine Learning for Compiler Optimization

Ashouri et al. provide a comprehensive survey of ML applied to compiler autotuning, pass ordering, and heuristic selection [3]. CompilerGym exposes LLVM optimization as a reinforcement-learning environment [4]. Other work learns graph representations of programs for heuristic tuning [5],[6]. MiniLang differs from these systems in scope and interpretability: optimization decisions occur at HIR level with explicit, low-dimensional features (14 integers), making it suitable for classroom demonstration and lightweight ML prototyping without requiring GPU-scale training infrastructure. The advisor is architected as a pluggable policy (heuristic today; ONNX or Python model tomorrow), aligning with survey recommendations for separating feature extraction from inference [3].

D. Educational Compilers

Educational compilers traditionally prioritize clarity over performance. MinLang extends the educational pattern by adding (1) a documented semantic analyzer with symbol tables (2) a serialized HIR dump format, and (3) quantitative evaluation tooling- bridging teaching and research.

III. The MiniLang Programming Language

MiniLang is intentionally small. It supports integer and boolean computation, functions conditionals, and loops. but omits pointers, arrays, help allocation, and I/O builtins.

A. Lexical Structure

Category	Rules
Whitespace	Space, tab, newline; tracked for source locations
Comments	`// to EOL; /* ... */ block comments`
Identifiers	`[A-Za-z_][A-Za-z0-9_]*`
Integers	`[0-9]+` (decimal, signed via unary -)
Keywords	`int, bool, void, if, else, while, return, true, false`
Operators	`+ - * / %`, comparisons, `&& || ! =`
Delimiters	`( ) { } [ ] ; ,`

TABLE I: MiniLang Lexical Rules

The lexer classifies keywords via classify_keyword() after scanning identifiers. Multi-character operators(==, !=, $\leq$, $\geq$, &&, ||) are recognized using one-character lookahead (match()).

B. Context-Free Grammar

MiniLang grammar is LL(1)-compatible:

```
program     ::= function*

function    ::= type IDENT "(" [ parameters ]
    ")" block

parameters  ::= type IDENT
                ( "," type IDENT )*

block       ::= "{" statement* "}"

statement   ::= block
              | type IDENT "=" expression ";"
              | "if" "(" expression ")"
                statement
                [ "else" statement ]
              | "while" "(" expression ")"
                statement
              | "return" [ expression ] ";"
              | expression ";"

expression  ::= logical_or

logical_or  ::= logical_and
                ( "||" logical_and )*

logical_and ::= equality
                ( "&&" equality )*

equality    ::= comparison
                ( ( "==" | "!=" ) comparison )
                    *

comparison  ::= term
                ( ( "<" | "<=" | ">" | ">=" )
                    term )*

term        ::= factor
                ( ( "+" | "-" ) factor )*

factor      ::= unary
                ( ( "*" | "/" | "%" ) unary )*
```

```
unary       ::= ( "!" | "-" ) unary
              | call

call        ::= primary

primary     ::= INT
              | "true"
              | "false"
              | IDENT [ "(" [ arguments ] ")"
                ]
              | "(" expression ")"

arguments   ::= expression
                ( "," expression )*

type        ::= "int" | "bool" | "void"
```

Operator precedence increases toward primary:logical OR (lowest) through multiplicative operators (highest). Unary operators bind tighter than binary operators.

C. Abstract Syntax Tree

The AST uses a tagged-union style with explicit Kind enumerations:

- Expressions: IntLiteral, BoolLiteral, Variable, Binary, Unary, Call.
- Statements: Block, VarDecl, If, While, Return, Expr
- Top level: ($Program \rightarrow vector$) of FunctionDecl

Each node carries a SourceLocation (filename, line, column) for diagnostics. Types are represented by TypeKind: Int, Bool, Void.

Example. The factorial program:

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

int main() {
    int x = factorial(5);
    return x;
}
```

parses into two FunctionDecl nodes with nested IfStmt, ReturnStmt, CallExpr. and VarDeclStmt nodes.

D. Static Semantics

The SemanticAnalyzer performs:
Pass 1 - Function registration: Build global map function_: ($name \rightarrow return\_type, parameter\_types, location$).Reject duplicate definitations.
Pass 2 - Per-function checking: Maintain stack of scope maps (scopes_) for block- enter/block-exit, Rules include:

Number equations consecutively. To make your equations more compact, you may use the solidus (/), the exp function, or appropriate exponents. Italicize Roman symbols for quantities and variables, but not Greek symbols. Use a long dash rather than a hyphen for a minus sign. Punctuate equations with commas or periods when they are part of a sentence, as in:

$$a + b = \gamma \tag{1}$$

Be sure that the symbols in your equation have been defined before or immediately following the equation. Use "(1)", not "Eq. (1)" or "equation (1)", except at the beginning of a sentence: "Equation (1) is . . ."

E. LaTeX-Specific Advice

Please use "soft" (e.g., `\eqref{Eq}`) cross references instead of "hard" references (e.g., `(1)`). That will make it possible to combine sections, add equations, or change the order of figures or citations without having to go through the file line by line.

Please don't use the `{eqnarray}` equation environment. Use `{align}` or `{IEEEeqnarray}` instead. The `{eqnarray}` environment leaves unsightly spaces around relation symbols.

Please note that the `{subequations}` environment in LaTeX will increment the main equation counter even when there are no equation numbers displayed. If you forget that, you might write an article in which the equation numbers skip from (17) to (20), causing the copy editors to wonder if you've discovered a new method of counting.

BibTeX does not work by magic. It doesn't get the bibliographic data from thin air but from .bib files. If you use BibTeX to produce a bibliography you must send the .bib files.

LaTeX can't read your mind. If you assign the same label to a subsubsection and a table, you might find that Table I has been cross referenced as Table IV-B3.

LaTeX does not have precognitive abilities. If you put a `\label` command before the command that updates the counter it's supposed to be using, the label will pick up the last counter to be cross referenced instead. In particular, a `\label` command should not go before the caption of a figure or a table.

Do not use `\nonumber` inside the `{array}` environment. It will not stop equation numbers inside `{array}` (there won't be any anyway) and it might stop a wanted equation number in the surrounding equation.

F. Some Common Mistakes

- The word "data" is plural, not singular.
- The subscript for the permeability of vacuum μ_0, and other common scientific constants, is zero with subscript formatting, not a lowercase letter "o".
- In American English, commas, semicolons, periods, question and exclamation marks are located within quotation marks only when a complete thought or name is cited, such as a title or full quotation. When quotation marks are used, instead of a bold or italic typeface, to highlight a word or phrase, punctuation should appear outside of the quotation marks. A parenthetical phrase or statement at the end of a sentence is punctuated outside of the closing parenthesis (like this). (A parenthetical sentence is punctuated within the parentheses.)

- A graph within a graph is an "inset", not an "insert". The word alternatively is preferred to the word "alternately" (unless you really mean something that alternates).
- Do not use the word "essentially" to mean "approximately" or "effectively".
- In your paper title, if the words "that uses" can accurately replace the word "using", capitalize the "u"; if not, keep using lower-cased.
- Be aware of the different meanings of the homophones "affect" and "effect", "complement" and "compliment", "discreet" and "discrete", "principal" and "principle".
- Do not confuse "imply" and "infer".
- The prefix "non" is not a word; it should be joined to the word it modifies, usually without a hyphen.
- There is no period after the "et" in the Latin abbreviation "et al.".
- The abbreviation "i.e." means "that is", and the abbreviation "e.g." means "for example".

An excellent style manual for science writers is [7].

G. Authors and Affiliations

The class file is designed for, but not limited to, six authors. A minimum of one author is required for all conference articles. Author names should be listed starting from left to right and then moving down to the next line. This is the author sequence that will be used in future citations and by indexing services. Names should not be listed in columns nor group by affiliation. Please keep your affiliations as succinct as possible (for example, do not differentiate among departments of the same organization).

H. Identify the Headings

Headings, or heads, are organizational devices that guide the reader through your paper. There are two types: component heads and text heads.

Component heads identify the different components of your paper and are not topically subordinate to each other. Examples include Acknowledgments and References and, for these, the correct style to use is "Heading 5". Use "figure caption" for your Figure captions, and "table head" for your table title. Run-in heads, such as "Abstract", will require you to apply a style (in this case, italic) in addition to the style provided by the drop down menu to differentiate the head from the text.

Text heads organize the topics on a relational, hierarchical basis. For example, the paper title is the primary text head because all subsequent material relates and elaborates on this one topic. If there are two or more sub-topics, the next level head (uppercase Roman numerals) should be used and, conversely, if there are not at least two sub-topics, then no subheads should be introduced.

I. Figures and Tables

a) Positioning Figures and Tables: Place figures and tables at the top and bottom of columns. Avoid placing them in the middle of columns. Large figures and tables may span across both columns. Figure captions should be below the figures; table heads should appear above the tables. Insert figures and tables after they are cited in the text. Use the abbreviation "Fig. 1", even at the beginning of a sentence.

TABLE II: Table Type Styles

Table Head	**Table Column Head**		
	Table column subhead	***Subhead***	***Subhead***
copy	More table copy[a]		

[a]Sample of a Table footnote.

Fig. 1: Example of a figure caption.

Figure Labels: Use 8 point Times New Roman for Figure labels. Use words rather than symbols or abbreviations when writing Figure axis labels to avoid confusing the reader. As an example, write the quantity "Magnetization", or "Magnetization, M", not just "M". If including units in the label, present them within parentheses. Do not label axes only with units. In the example, write "Magnetization (A/m)" or "Magnetization {A[m(1)]}", not just "A/m". Do not label axes with a ratio of quantities and units. For example, write "Temperature (K)", not "Temperature/K".

ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

Please number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use "Ref. [3]" or "reference [3]" except at the beginning of a sentence: "Reference [3] was the first ..."

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors' names; do not use "et al.". Papers that have not been published, even if they have been submitted for publication, should be cited as "unpublished" [4]. Papers that have been accepted for publication should be cited as "in press" [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

REFERENCES

[1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.

[2] J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

[3] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[4] K. Elissa, "Title of paper if known," unpublished.

[5] R. Nicole, "Title of paper with only first word capitalized," J. Name Stand. Abbrev., in press.

[6] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, "Electron spectroscopy studies on magneto-optical media and plastic substrate interface," IEEE Transl. J. Magn. Japan, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetics Japan, p. 301, 1982].

[7] M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.